

Product Video Discovery Dashboard

Node.js backend · React frontend · Instagram Reels + Meta Ad Library

For any queries, contact: **Ansh Chaudhary** | AI Automation & Developer | **+91 7830147870**

Overview

Build a full-stack dashboard where a user types a product name or pastes a product link and gets at least 40 relevant short-form videos back: 20 from Instagram Reels and 20 from the Meta Ad Library. An image-analysis "brain" reads the product's photo so the videos you return show the exact product, not just similar keywords.

This assignment tests how you design automation pipelines, handle unreliable external data sources, and ship a working full-stack product. Treat it as a small real project: the result should run end to end, not just look good in a demo.

Tech stack

The backend must be Node.js and the frontend must be React. Everything else is your choice, and we want to see why you chose it.

Layer	Requirement	Notes
Backend	Node.js (Express, Fastify or NestJS)	REST or WebSocket/SSE API
Frontend	React (Vite or Next.js)	Any UI library is fine
Data store	Your choice (PostgreSQL, MongoDB, Redis, SQLite)	Needed for search history and de-duplication
Image analysis	Any vision model or API	Justify the choice in your README
Video sources	Instagram Reels, Meta Ad Library	TikTok is optional

Use environment variables for every key and token. Never commit secrets to the repository.

Core requirements

The dashboard accepts two kinds of input, and both must work.

1. **Product name or keyword.** Example: "oversized graphic tee" or "protein dark chocolate".
2. **Product link.** Example: a Shopify, Amazon or brand-site product URL. The system must open the page, extract the product title, description and main image, and use them as the search context.

For each search, the system must:

- Collect at least 20 Instagram Reels and at least 20 Meta Ad Library videos that relate to that product.
- Use the product image to check that the videos show the exact product or a very close visual match.

- Return results with a thumbnail, source platform, video link, caption or ad copy, and a match score.
- Never show the same video twice within one result set, and avoid repeating videos from earlier searches (see the uniqueness section).

Video sourcing

Every search must return a minimum of 40 videos, split across two required sources and one optional source.

Source	Minimum per search	Status
Instagram Reels	20	Required
Meta Ad Library (video ads)	20	Required
TikTok	No minimum	Optional bonus

How you collect the videos is up to you: official APIs, the Meta Ad Library API, a scraping or data provider, or browser automation. Whatever you choose, your README must explain:

- Which method you used for each source and why.
- How the system handles rate limits, blocked requests, login walls and missing data.
- What happens when a source returns fewer than 20 usable videos (for example, widening the query or flagging the shortfall in the UI instead of failing silently).

If you add TikTok, keep it behind its own toggle so it never blocks the two required sources.

Image-analysis brain

The brain's job is to make sure a returned video shows the exact product, not just a product in the same category. It must analyse the product image first, then use that analysis to search and to rank results.

Minimum behaviour:

- Extract visual attributes from the product image: product type, colours, prints or graphics, logos, text on the product, material and shape.
- Turn those attributes into better search queries and hashtags for each platform.
- Compare video frames or thumbnails against the product image and give every video a match score from 0 to 100.
- Discard or clearly mark videos below a match threshold that you define and document.
- Show the reason for each score in the UI (for example, "same print and colour, worn by a model").

How you build it is open. Vision-language models, image embeddings with similarity search, or a hybrid are all acceptable. We will judge it on accuracy, so test with at least 5 different products and include your results in the README.

Unique results on every search

Each new search must surface videos the user has not seen before. The same video should not keep coming back across searches, even when the queries overlap.

Requirements:

- Store a unique ID for every video returned (platform video ID or a hash of the media URL) together with the search that returned it.
- Before showing results, filter out videos already returned in earlier searches, and also remove duplicates and near-duplicates inside the same search (reposts, re-uploads, the same ad running under several IDs).
- If de-duplication leaves fewer than 20 videos for a source, fetch more using expanded queries, related hashtags or deeper pagination until the minimum is met.
- Provide a "Show previously seen" toggle so a user can still open old results deliberately.
- Explain your de-duplication strategy in the README, including how you detect near-duplicates.

Dashboard UI

The frontend is a React single-page dashboard. It should be clean and usable, but working features matter more than visual polish.

- **Search bar** that accepts a product name or a product URL, with an optional image upload as a third input.
- **Product panel** showing the extracted title, main image and the attributes the brain detected.
- **Results grid** with separate tabs or sections for Instagram Reels and Meta Ad Library (and TikTok if built), each with a live count against the 20-video minimum.
- **Video cards** with thumbnail, inline playback or a link to the original, caption or ad copy, platform badge and match score.
- **Progress state** so the user sees what the pipeline is doing (fetching page, analysing image, searching Instagram, searching Meta, scoring).
- **Filters and sorting** by match score, platform and newest first.
- **Search history** listing earlier searches and their results.
- **Error and empty states** that explain what went wrong and what the user can try next.
- Responsive layout that works on desktop and tablet.

Backend and API

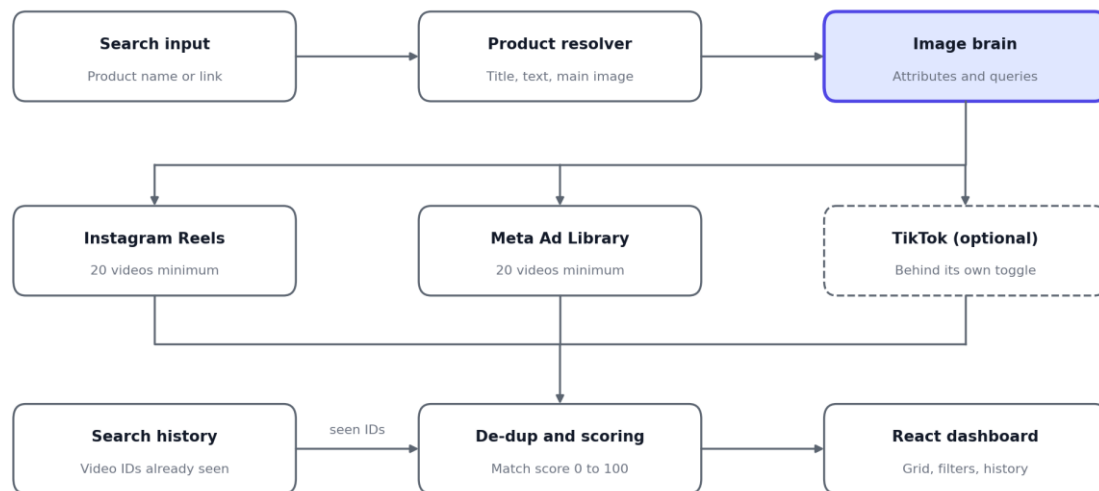
The Node.js backend runs one pipeline per search. The diagram at the end of this section is a suggested shape; you may design your own if you can explain it.

Backend requirements:

- Expose a clean API, for example `POST /api/search`, `GET /api/search/:id` and `GET /api/history`, using REST plus SSE or WebSockets for live progress.
- Run Instagram and Meta collection in parallel, with timeouts, retries and per-source error handling so one failing source never breaks the whole search.
- Use a job queue or background worker for long searches instead of blocking the request.
- Cache product-page and image analysis so the same product link is not re-processed every time.

- Validate all input and block unsafe URLs (for example internal addresses) before fetching any product page.
- Keep credentials in environment variables and respect each platform's terms, rate limits and data rules.
- Add structured logging and clear error responses, plus basic tests for the de-duplication and scoring logic.

The image brain shapes every query before videos are collected



Suggested pipeline. The image brain feeds every source collector; results are de-duplicated against search history before reaching the dashboard.

Deliverables and submission

Submit the following when you are done:

1. **Source code** in a Git repository (GitHub or GitLab link) with a clear commit history, separate backend/ and frontend/ folders and a .env.example file.
2. **README** covering setup steps, architecture, how each video source is collected, how the image brain works, the de-duplication strategy, known limitations and what you would build next.
3. **Demo video** of 3 to 5 minutes showing a search by product name, a search by product link, the 20 + 20 minimum being met, and a repeat search returning new videos.
4. **Test evidence**: results for at least 5 different products, with the counts per source and a few examples of good and bad matches.
5. **Live link** (optional) if you deploy it anywhere.

Do not include real API keys, tokens or personal account credentials in the repository or the demo.

Evaluation criteria

We will run your project ourselves and score it out of 100. Suggested time to complete: 5 days.

Area	Weight	What we look for
Video sourcing	25	20 Instagram Reels and 20 Meta videos returned reliably per search
Image-analysis brain	25	Videos show the exact product; match scores are explained and sensible
Uniqueness and de-duplication	15	New searches give new videos; no duplicates inside a result set
Backend design (Node.js)	15	Clean structure, parallel collection, error handling, caching, security
Dashboard (React)	10	Usable flow, live progress, filters, clear empty and error states
Documentation and demo	10	README quality, honest limitations, working demo video

Bonus points: TikTok as a third source, saving or exporting a shortlist of videos, and a Docker setup that runs the whole project with one command.

We value a working, honest solution over a perfect-looking one. If a source blocks you or a limit stops you from hitting a target, tell us what you tried and what you would do next.

Questions?

For any queries, contact: **Ansh Chaudhary** | AI Automation & Developer | **+91 7830147870**